# ML-KEM C11 — Public API Usage Guide

**Implementation-specific developer documentation**
**Interface:** ml_kem.h | **Public functions:** 9 | **Parameter sets:** ML-KEM-512, ML-KEM-768, ML-KEM-1024
**Ports included in the supplied sources:** Linux userspace, Linux kernel, FreeRTOS
**Document date:** November 3, 2026

This guide describes the actual public interface and observable behavior of the supplied ML-KEM C implementation. It focuses on application integration: how to create key/decapsulation contexts, export and import keys, encapsulate with or without a reusable pool, decapsulate ciphertexts, and release resources. The API implements operations based on **FIPS 203**; this document does not represent independent security certification or FIPS validation.

## 1. Start here

Include ml_kem.h in the calling code. Compile the cryptographic source files, exactly one platform port, and the application with the matching port selection macro. The interface has two independently managed object types:

- **Key and decapsulation object** — owns one ML-KEM key pair and a reusable, fixed-size decapsulation slot pool. Created by ml_kem_create_object() or ml_kem_create_object_from_imported_sk(); destroyed by ml_kem_destroy_core().
- **Optional encapsulation pool** — owns reusable encapsulation slots, partitioned by ML-KEM parameter set. Created by create_encaps_pool(); destroyed by ml_kem_encaps_pool_destroy(). Pass NULL to ml_kem_encaps_core() instead if a temporary encapsulation context is acceptable.

**Typical flow:** create keys → export public key → encapsulate using that public key → send ciphertext → decapsulate with the matching key object → both parties obtain a 32-byte shared secret → clean up. A stored secret key can be exported and later imported to reconstruct a key/decapsulation object.

## 2. Parameters and sizes

Use the constants from ml_kem_core_header.h. The public API expects the listed *exact* lengths for public keys, ciphertexts, and shared secrets in the relevant calls.

| Parameter set | Enum value | Public key (bytes) | Secret key (bytes) | Ciphertext (bytes) | Shared secret (bytes) |
|---|---|---|---|---|---|
| ML-KEM-512 | ML_KEM_512 | 800 | 1,632 | 768 | 32 |
| ML-KEM-768 | ML_KEM_768 | 1,184 | 2,400 | 1,088 | 32 |
| ML-KEM-1024 | ML_KEM_1024 | 1,568 | 3,168 | 1,568 | 32 |

Constant families: RES_PUBL_PART_LVL_512/768/1024, LEN_SK_512/768/1024, LEN_CIPHERTEXT_512/768/1024, and ML_KEM_SEED_BYTES (32). Maximum pool capacity: ML_KEM_MAX_POOL_SLOTS (128). The allowed slot count is **1–128** for a key/decapsulation object and **0–128 per level** for the encapsulation pool, with at least one encapsulation subpool enabled.

## 3. All nine public functions

| Function | Main purpose | Return on success |
|---|---|---|
| ml_kem_create_object() | Generate a key pair and a decapsulation pool | Context pointer |
| ml_kem_create_object_from_imported_sk() | Reconstruct a key/decapsulation object from serialized secret key | Context pointer |
| ml_kem_destroy_core() | Destroy key/decapsulation object | void |
| ml_kem_encaps_core() | Encapsulate; optional encapsulation pool | 0 |
| ml_kem_decaps_core() | Decapsulate with a key/decapsulation object | 0 |
| ml_kem_get_public_key() | Copy serialized public key to caller buffer | 0 |
| ml_kem_get_sk() | Export serialized secret key | Number of bytes written |
| create_encaps_pool() | Allocate encapsulation slots for selected levels | Pool pointer |
| ml_kem_encaps_pool_destroy() | Destroy encapsulation pool | void |

### 3.1 Create a key and decapsulation context

```
struct ml_kem_general_keys_decaps_ctx *ml_kem_create_object(
    enum ml_kem_k level, size_t ml_kem_pool_count,
    ml_kem_entropy_fn entropy);
```

level selects ML_KEM_512, ML_KEM_768, or ML_KEM_1024. ml_kem_pool_count reserves the maximum number of simultaneous decapsulation operations for the generated key pair (1–128). entropy is a callback that fills a buffer with cryptographically secure random bytes and returns zero on success; pass NULL to use the selected port's default source. On Linux userspace and the supplied FreeRTOS port, allocation/initialization failure yields NULL. **Linux kernel callers must handle an ERR_PTR(...) error result** rather than assuming all failures are NULL.

### 3.2 Import a serialized secret key

```
struct ml_kem_general_keys_decaps_ctx *
ml_kem_create_object_from_imported_sk(
    u8 *sk, size_t size_sk, size_t size_pool);
```

sk points to a serialized ML-KEM secret key and size_sk must be exactly 1,632, 2,400, or 3,168 bytes; the length identifies the parameter set. size_pool sets the new decapsulation slot count (1–128). The importer copies key material and validates the stored hash of the public key and encoded public-key coefficients. It is **not** a substitute for all possible validation of arbitrary untrusted private-key material. On Linux userspace/FreeRTOS failure returns NULL; in the Linux kernel it can return ERR_PTR(...). Destroy the resulting object with ml_kem_destroy_core().

### 3.3 Destroy a key and decapsulation context

```
void ml_kem_destroy_core(
    struct ml_kem_general_keys_decaps_ctx *ctx);
```

Releases the decapsulation pool and persistent key material. NULL is accepted. The caller must ensure that no thread is still using the object, its slots, or its key material during destruction. The caller remains responsible for clearing any separately exported secret-key buffers.

### 3.4 Encapsulate (pool or standalone)

```
int ml_kem_encaps_core(
    struct ml_kem_pool_encaps_ctx *ctx_pool,
    u8 *pk, size_t size_pk,
    u8 *buffer_ct, size_t size_ct,
    u8 *buffer_ss, size_t size_ss,
    ml_kem_entropy_fn entropy);
```

pk is the serialized recipient public key. buffer_ct receives the ciphertext and buffer_ss receives the 32-byte shared secret. The parameter set is selected from the *combination* of size_pk, size_ct, and size_ss, so their values must correspond to a single row of the sizes table. ctx_pool == NULL creates and destroys temporary encapsulation state for the call; a non-NULL pool acquires a reusable slot for the selected level. The pool must contain a subpool for that level. Returns 0 on success, a negative error code otherwise, including -EBUSY when all matching slots are taken. Output buffers should be treated as valid **only after success**. The public wrapper does not perform comprehensive validation of an arbitrary supplied public key's encoding; validate externally received keys as required by the application/protocol.

### 3.5 Decapsulate

```
int ml_kem_decaps_core(
    struct ml_kem_general_keys_decaps_ctx *ctx,
    u8 *ciphertext, size_t len_ciphertext,
    u8 *result, size_t len_result);
```

ctx is the key/decapsulation object containing the recipient secret key. ciphertext has the parameter-set-specific exact length; result must have **exactly 32 bytes** (len_result == ML_KEM_SEED_BYTES). The function atomically acquires a decapsulation slot, decrypts, reconstructs an expected ciphertext, and chooses the real or fallback shared secret. Returns 0 for a successfully executed operation, including the **implicit-rejection** case of a well-sized but cryptographically invalid ciphertext. It does not return a separate "invalid ciphertext" status. Wrong pointers/lengths or an unavailable slot produce errors such as -EINVAL or -EBUSY.

### 3.6 Export the public key

```
int ml_kem_get_public_key(
    struct ml_kem_general_keys_decaps_ctx *ctx,
    u8 *buffer_for_public_key, size_t size_buffer);
```

Copies the serialized public key out of the key object. size_buffer must match the public-key size **exactly**, rather than merely being large enough. Returns 0 on success or a negative error code on failure. The public key can be distributed to encapsulating peers through an authenticated protocol.

### 3.7 Export the secret key

```
int ml_kem_get_sk(
    struct ml_kem_general_keys_decaps_ctx *ctx,
    u8 *buffer_for_sk, size_t size_buffer_for_sk);
```

Exports the serialized secret key (the implementation writes the packed secret vector, serialized public key, stored public-key hash, and fallback value z). size_buffer_for_sk must be **at least** the corresponding LEN_SK_* size. Returns the **number of bytes actually written** (1,632 / 2,400 / 3,168) on success, otherwise a negative error code. This buffer is highly sensitive: protect it during transport/storage and securely erase temporary copies.

### 3.8 Create an encapsulation pool

```
struct ml_kem_pool_encaps_ctx *create_encaps_pool(
    size_t number_slots_512,
    size_t number_slots_768,
    size_t number_slots_1024);
```

Each argument reserves 0–128 reusable encapsulation slots for the named parameter set. A zero value disables that subpool; all three values cannot be zero. For example, `create_encaps_pool(2, 0, 4)` provides two simultaneous ML-KEM-512 encapsulations and four ML-KEM-1024 encapsulations, but none for ML-KEM-768. A single pool can be reused across different recipient public keys. On Linux userspace/FreeRTOS creation failure returns `NULL`; Linux kernel callers must check `IS_ERR()` as appropriate.

### 3.9 Destroy an encapsulation pool

```
void ml_kem_encaps_pool_destroy(
    struct ml_kem_pool_encaps_ctx *ctx);
```

Destroys the pool and its allocated per-slot working buffers. `NULL` is accepted. Call it only after all current encapsulation operations using that pool have finished.

## 4. Complete Linux userspace example

This example exercises **all nine public API functions** with ML-KEM-512, checks that encapsulation and decapsulation agree, and uses a reusable encapsulation pool. To encapsulate without a pool, pass NULL as the first argument to ml_kem_encaps_core(). It uses the Linux userspace default entropy source by passing NULL.

```c
// Linux userspace: use all nine public API functions (ML-KEM-512).
#include <stdio.h>
#include <string.h>
#include "ml_kem.h"

int main(void)
{
    struct ml_kem_general_keys_decaps_ctx *keys = NULL, *imported = NULL;
    struct ml_kem_pool_encaps_ctx *pool = NULL;
    u8 pk[RES_PUBL_PART_LVL_512], sk[LEN_SK_512];
    u8 ct[LEN_CIPHERTEXT_512];
    u8 ss_enc[ML_KEM_SEED_BYTES], ss_dec[ML_KEM_SEED_BYTES];
    int ok = 0;

    keys = ml_kem_create_object(ML_KEM_512, 1, NULL);
    if (!keys) goto cleanup;
    if (ml_kem_get_public_key(keys, pk, sizeof(pk))) goto cleanup;
    if (ml_kem_get_sk(keys, sk, sizeof(sk)) != (int)sizeof(sk)) goto cleanup;

    imported = ml_kem_create_object_from_imported_sk(sk, sizeof(sk), 1);
    if (!imported) goto cleanup;
    pool = create_encaps_pool(1, 0, 0);
    if (!pool) goto cleanup;

    if (ml_kem_encaps_core(pool, pk, sizeof(pk), ct, sizeof(ct),
                           ss_enc, sizeof(ss_enc), NULL)) goto cleanup;
    if (ml_kem_decaps_core(imported, ct, sizeof(ct),
                           ss_dec, sizeof(ss_dec))) goto cleanup;
    if (memcmp(ss_enc, ss_dec, sizeof(ss_enc))) goto cleanup;
    ok = 1;

cleanup:
    ml_kem_encaps_pool_destroy(pool);
    ml_kem_destroy_core(imported);
    ml_kem_destroy_core(keys);
    ml_kem_memzero(sk, sizeof(sk));
    ml_kem_memzero(ss_enc, sizeof(ss_enc));
    ml_kem_memzero(ss_dec, sizeof(ss_dec));
    puts(ok ? "PASS" : "FAIL");
    return ok ? 0 : 1;
}
```

The ml_kem_memzero() calls are platform-port helpers rather than part of the nine-function public interface. The example has been compiled and executed against the supplied userspace source files, producing PASS.

### Compile the example

Place ml_kem_api_guide_example.c beside the supplied portable *.c/*.h core sources and ml_kem_userspace_port.c/.h, then run the following from that directory:

```
cc -O2 -std=c11 -Wall -Wextra -DUSERSPACE -DFULL_MATRIX=0 -I. \
  ml_kem.c ml_kem_create_keys.c ml_kem_decaps_pool.c \
  ml_kem_decrypt.c ml_kem_encaps.c ml_kem_encaps_pool.c \
  ml_kem_ntt_main.c ml_kem_sha3.c ml_kem_shake.c \
  keccak_1600.c ml_kem_userspace_port.c \
  ml_kem_api_guide_example.c -o ml_kem_api_example
./ml_kem_api_example
```

Expected output: PASS. This example is a **functional integration check**, not a KAT, timing-leakage test, or security certification.

## 5. Error handling and lifecycle

### Integer-returning functions

| Return | Meaning in this implementation |
|---|---|
| 0 | Successful encapsulation, decapsulation, or public-key export |
| Positive secret-key length | Successful ml_kem_get_sk() export |
| -EINVAL / -ML_KEM_EINVAL | Invalid pointer, size, security level, or configuration |
| -ENOMEM / -ML_KEM_ENOMEM | Allocation failure where applicable |
| -EBUSY | No reusable slot currently free; caller may retry later |
| -ML_KEM_SYSTEM_ENTROPY_FAILED | Default entropy source failed |
| -ML_KEM_CALLBACK_ENTROPY_FAILED | User-supplied entropy callback failed |

These are possible values depending on the call and port, not a promise that every function returns every listed value. In Linux userspace, failed object/pool creation is reported as NULL (without preserving an error code through the pointer). In the kernel port, creation helpers can return ERR_PTR(-errno) and must be checked using kernel pointer-error conventions.

### Resource ownership and concurrency

- Every successful ml_kem_create_object() or import has exactly one matching ml_kem_destroy_core() call; every successful create_encaps_pool() has one matching ml_kem_encaps_pool_destroy() call.
- A reusable pool reserves workspace ahead of time. Slot acquisition uses atomics and returns -EBUSY rather than blocking when no slot is available.
- The decapsulation pool is associated with **one** key pair. The encapsulation pool may encapsulate to **different** public keys, sharing reusable per-level workspace.
- It is safe by design to perform operations on separate acquired slots concurrently. Callers must nevertheless avoid shared mutable output buffers and must not destroy or reconfigure contexts while operations are active.
- ml_kem_encaps_core(NULL, ...) allocates a temporary context per operation; use a pool when repeated operations should avoid those allocations.
- Output keys and ciphertexts belong to caller-provided buffers. Wipe secret copies in the application when no longer needed.

## 6. Platform notes

**Linux userspace (-DUSERSPACE).** Include ml_kem_userspace_port.c/.h. The supplied port uses malloc/free, Linux getrandom() for default entropy, C11 atomics, and a volatile-byte memory wipe. The example in Section 4 targets this port. A NULL entropy callback is valid here.

**Linux kernel (-DLINUX_KERNEL).** Include ml_kem_linux_kernel_port.c/.h and use a kernel-module build environment and kernel headers rather than the userspace compiler command. The port uses kvzalloc(..., GFP_KERNEL), kvfree, get_random_bytes, memzero_explicit, and kernel atomics. Creation failures can return ERR_PTR values, so use IS_ERR()/PTR_ERR() appropriately. The API exports nine GPL symbols; allocation may sleep, so call contexts must permit GFP_KERNEL allocations where they occur.

**FreeRTOS (-DFREERTOS).** Include ml_kem_freertos_port.c/.h. Select the backend with ML_KEM_ATOMIC_STDATOMIC **or** ML_KEM_ATOMIC_FREERTOS. The port uses pvPortMalloc/vPortFree. Its default ml_kem_entropy() intentionally fails; supply a working cryptographically secure ml_kem_entropy_fn callback to operations that generate fresh randomness. Build and port validation depend on the selected board, FreeRTOS configuration, and entropy device.

**Allocation alignment.** Internal Keccak state buffers are used as u64 lanes. When adapting an allocator/port, preserve alignment suitable for u64 (at least 8 bytes on the supported targets), including the offsets of any sliced buffers. Validate the layout on the target ABI.

## 7. Security and integration notes

1. **Implicit rejection is not an API error.** With a correctly sized but invalid ciphertext, decapsulation deliberately computes a fallback shared secret. A successful return indicates the decapsulation procedure completed; it does not authenticate the peer or prove the ciphertext was originally valid.
2. **Authenticate public keys at the protocol level.** ML-KEM by itself provides no peer identity binding. The wrapper checks size combinations but does not expose comprehensive canonicality checks for arbitrary externally supplied public keys before encapsulation.
3. **Treat serialized secret keys as sensitive.** Protect exported sk at rest/in transit, wipe temporary copies, and do not log seeds or shared secrets.
4. **Entropy quality is essential.** Custom callbacks must supply unpredictable cryptographic randomness; never replace them with a deterministic PRNG outside controlled test builds.
5. **Build configuration matters.** FULL_MATRIX=1 enables precomputation in the decapsulation path, trading memory for runtime work. ML_KEM_TEST and ML_KEM_DUDECT_TEST expose test-specific behavior and should not be used in production builds.
6. **Constant-time claims require separate validation.** The source uses intended constant-time selection in the decapsulation path, but resistance to timing leakage depends on the platform, compiler, and final machine code. Run appropriate KAT, sanitizer, concurrency, and dudect tests for the deployed configuration.

## 8. Source files and further reference

Public interface: ml_kem.h. Shared structures, constants, callback type, and internal declarations: ml_kem_core_header.h. Core entry points: ml_kem.c. Key generation: ml_kem_create_keys.c. Encapsulation: ml_kem_encaps.c and ml_kem_encaps_pool.c. Decryption and decapsulation pool: ml_kem_decrypt.c and ml_kem_decaps_pool.c. Arithmetic and hashing: ml_kem_ntt_main.c, ml_kem_sha3.c, ml_kem_shake.c, and keccak_1600.c. Platform adapters: ml_kem_userspace_port.*, ml_kem_linux_kernel_port.*, and ml_kem_freertos_port.*.

**Normative algorithm reference:** NIST FIPS 203, *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (August 2024), https://doi.org/10.6028/NIST.FIPS.203. **Scope of this guide:** public API and source-level integration behavior of the files supplied for this project.